



UNIVERSITÉ DE MONTPELLIER

Rapport du TP1 de Machine Learning 2

Massila LAKAF, Imane LAZIZI, Kanzy YOUSSEF

*Encadrant : M.Pascal Poncelet
2025-2026*

Sommaire

1	Génération des données avec <code>make_moons</code>	1
1.1	Paramètres de génération	1
2	Processus d'apprentissage	1
2.1	Division des données	1
2.2	Phase d'entraînement	1
2.3	Phase de test	1
2.4	Phase de production	2
3	Analyse exploratoire des données	2
3.1	Dimensions des données	2
3.2	Distribution des classes	2
3.3	Affichage sous forme de dictionnaire	2
3.4	Visualisation graphique	3
4	Division et normalisation des données	3
4.1	Division des données avec <code>train_test_split</code>	3
4.1.1	Paramètres de division	3
4.1.2	Résultats de la division	3
4.2	Normalisation des données	4
4.2.1	Création du normaliseur	4
4.2.2	Qu'est-ce que normaliser ?	4
4.3	Le <i>Data Leakage</i> : un point crucial en Machine Learning	4
4.3.1	Principe fondamental	4
4.3.2	Application de la normalisation	4
4.4	Application de la normalisation avec <code>transform</code>	4
4.4.1	Formule de normalisation	4
5	Définition et entraînement du modèle	5
5.1	Architecture du réseau de neurones	5
5.1.1	Fonctionnement du réseau	5
5.1.2	Choix des fonctions d'activation	6
5.1.3	Choix de la couche Dense et notation fonctionnelle	6
5.2	Structure du modèle : <code>model.summary()</code>	7
5.3	Compilation du modèle	7
5.3.1	Paramètres de compilation	7
5.4	Entraînement du modèle : premier essai	8
5.5	Test avec 10 epochs	8
5.6	Introduction du <code>batch_size</code>	8
5.6.1	Qu'est-ce que le <code>batch_size</code> ?	9
5.6.2	Impact du <code>batch_size</code> sur les courbes	9
5.7	Prédiction et évaluation	9
5.7.1	Problème rencontré	10
5.7.2	Solution : conversion probabilité $\rightarrow$ classe	10

1 Génération des données avec `make_moons`

```
1 X, y = make_moons(n_samples=3000, noise=0.25, random_state=42)
```

La fonction `make_moons` permet de générer des données synthétiques pour l'apprentissage. Le nombre total de données générées est 3000.

- `X` : Reçoit les données d'entrée (coordonnées des points dans l'espace 2D)
- `y` : Reçoit les vraies classes de chaque point (0 ou 1)

1.1 Paramètres de génération

Le paramètre `noise` (bruit) : Il représente le niveau de perturbation aléatoire appliqué aux données. Par exemple, un point théorique $p(0.2, 1.2)$ peut être légèrement modifié pour devenir $p(0.18, 1.23)$. Ici, le bruit d'erreur est de 0.25, ce qui rend le problème plus réaliste et plus difficile.

Le paramètre `random_state` : On lui donne n'importe quel nombre entier (42 par convention, en référence au *Guide du voyageur galactique*). Il sert à fixer la graine aléatoire pour avoir toujours les mêmes données à chaque exécution, assurant ainsi la reproductibilité des résultats.

2 Processus d'apprentissage

2.1 Division des données

Les 3000 données sont divisées en deux ensembles distincts :

- **Données d'entraînement (`train`)** : Par exemple 80% des données (2400 points)
- **Données de test (`test`)** : Par exemple 20% des données (600 points)

2.2 Phase d'entraînement

Sur les données d'entraînement (les 80%), le modèle effectue plusieurs **tours** (appelés *epochs*). Par exemple, avec 30 tours :

1. **À chaque tour**, pour chaque point des 2400 :
 - Le modèle **prédit** une classe
 - Il **compare** sa prédiction avec la vraie classe
 - Il **ajuste** sa formule de prédiction (les poids du réseau)
2. Le modèle passe ensuite au tour suivant
3. Au fil des tours, le modèle devient de plus en plus performant
4. Au 30^{ème} tour, sa formule de prédiction est beaucoup plus efficace qu'au début

2.3 Phase de test

Après l'entraînement, vient la phase de test pour évaluer les performances :

- Le modèle prend les `X_test` (coordonnées jamais vues pendant l'entraînement)
- Il produit des prédictions `y_pred`
- On compare `y_pred` avec les vraies classes `y_test` qu'on avait mises de côté
- Cela donne un **score de fiabilité** (accuracy)

Important : Le modèle **ne s'ajuste pas** pendant l'étape de test. L'étape de test sert uniquement à donner un score pour évaluer la performance sur des données jamais vues.

2.4 Phase de production

Finalement, le modèle va être utilisé en production sur de vraies nouvelles données :

- On a uniquement les X (coordonnées)
- On n'a **pas** les résultats y pour vérifier ou ajuster
- Le modèle prédit de manière autonome

3 Analyse exploratoire des données

```
1 print("X shape:", X.shape, " | y shape:", y.shape)
2 print("Un exemple des donn es dans X", X[0])
3 print("Un exemple des donn es dans y", y[0])
4
5 u, c = np.unique(y, return_counts=True)
6 print("Distributions des classe:", X.shape,
7       dict(zip(map(int, u), map(int, c))))
8
9 plt.figure(figsize=(6,6))
10 plt.scatter(X[:, 0], X[:, 1], c=y, s=12)
11 plt.title("make_moons")
12 plt.xlabel("x1")
13 plt.ylabel("x2")
14 plt.tight_layout()
15 plt.show()
```

3.1 Dimensions des données

$X.shape$ nous donne le nombre de données d'entrée ainsi que leurs dimensions. Ici :

- $X.shape = (3000, 2)$ signifie 3000 exemples
- Chaque exemple possède 2 **features** (caractéristiques) : les coordonnées x_1 et x_2 du point dans l'espace 2D

$y.shape$ nous donne le nombre de sorties. Ici :

- $y.shape = (3000,)$ signifie 3000 sorties
- Chaque sortie est la classe du point correspondant (0 ou 1)
- y n'a qu'une seule dimension car la sortie est scalaire (soit 0, soit 1)

3.2 Distribution des classes

$u, c = np.unique(y, return_counts=True)$:

Cette fonction analyse la distribution des classes dans les données :

- u : Reçoit un tableau contenant toutes les valeurs uniques (sans doublons) présentes dans y . Dans notre cas : $[0, 1]$
- c : Reçoit un tableau du même ordre que u , où chaque élément à l'indice i représente le nombre de répétitions de la valeur $u[i]$

Exemple : Si $u = [0, 1]$ et $c = [1500, 1500]$, cela signifie qu'il y a 1500 points de classe 0 et 1500 points de classe 1.

3.3 Affichage sous forme de dictionnaire

$dict(zip(map(int, u), map(int, c)))$:

Cette expression transforme les tableaux en dictionnaire pour un affichage plus lisible :

1. $map(int, u)$ et $map(int, c)$: Convertissent les éléments en entiers

2. `zip(...)` : Associe chaque valeur de `u` avec son nombre de répétitions dans `c`, créant des paires
3. `dict(...)` : Transforme ces paires en dictionnaire

Résultat affiché : {0: 1500, 1: 1500}

Cela signifie : classe 0 apparaît 1500 fois, classe 1 apparaît 1500 fois.

3.4 Visualisation graphique

`plt.scatter(X[:, 0], X[:, 1], c=y, s=12)` :

Cette commande crée un nuage de points dans un repère 2D :

- `X[:, 0]` : Coordonnées x_1 de tous les points (abscisses)
- `X[:, 1]` : Coordonnées x_2 de tous les points (ordonnées)
- `c=y` : Coloration des points selon leur classe (classe 0 dans une couleur, classe 1 dans une autre)
- `s=12` : Taille de chaque point dans le graphique

Les autres lignes `plt` : Elles servent à configurer l’affichage du graphique (titre, labels des axes, ajustement de la mise en page, et affichage final).

4 Division et normalisation des données

```

1 X_train, X_test, y_train, y_test = train_test_split(
2     X, y, test_size=0.2, random_state=42, stratify=y
3 )
4
5 scaler = StandardScaler()
6 scaler.fit(X_train)
7 X_train = scaler.transform(X_train)
8 X_test = scaler.transform(X_test)

```

4.1 Division des données avec `train_test_split`

La fonction `train_test_split` divise les données en données d’entraînement (train) et données de test (test).

4.1.1 Paramètres de division

- `test_size=0.2` : 20% des données sont allouées au test, donc 80% restent pour l’entraînement
- `random_state=42` : Sert à avoir toujours la même division des données à chaque exécution (reproductibilité)
- `stratify=y` : Garantit que les proportions des classes sont les mêmes dans train et test que dans les données originales. Ici, si on a 50% de classe 0 et 50% de classe 1 dans les données totales, on aura aussi 50/50 dans train et dans test

4.1.2 Résultats de la division

En résumé, après cette division :

- `X_train` : Reçoit les features (coordonnées) des données d’entraînement
- `X_test` : Reçoit les features des données de test
- `y_train` : Reçoit les classes (résultats) des données d’entraînement
- `y_test` : Reçoit les classes des données de test

4.2 Normalisation des données

4.2.1 Création du normaliseur

```
1 scaler = StandardScaler()
```

Cette ligne crée un objet `scaler` qui permet de normaliser les données.

4.2.2 Qu'est-ce que normaliser ?

Normaliser signifie ramener les features des données autour de 0. Plus précisément :

- La **moyenne** de chaque feature devient 0

- L'**écart-type** de chaque feature devient 1

Intérêt de la normalisation : C'est pour que le modèle ne se concentre pas uniquement sur une seule feature et ne néglige pas les autres. Par exemple :

- Feature f_1 a un intervalle $[1, 2]$

- Feature f_2 a un intervalle $[10000, 20000]$

Sans normalisation, le modèle peut prendre en considération principalement f_2 (car ses valeurs sont beaucoup plus grandes) et ignorer f_1 . Pour éviter ce biais, on normalise toutes les features à la même échelle.

4.3 Le *Data Leakage* : un point crucial en Machine Learning

4.3.1 Principe fondamental

Pour éviter le *data leakage* (fuite de données), lorsqu'on fait :

```
1 scaler.fit(X_train)
```

On le fait **uniquement** sur les données d'entraînement. Cette opération calcule la **moyenne** μ et l'**écart-type** σ de chaque feature uniquement à partir des données train.

4.3.2 Application de la normalisation

Ensuite, on utilisera cette moyenne μ et cet écart-type σ pour normaliser :

- Les données d'entraînement (`X_train`)

- Les données de test (`X_test`)

Important : Ne pas inclure les données de test dans le calcul de la moyenne et de l'écart-type permet au modèle d'être complètement ignorant des données de test. Ainsi, les résultats obtenus sur `y_test` seront beaucoup plus fiables et représentatifs de la vraie performance du modèle en production.

4.4 Application de la normalisation avec transform

La normalisation se fait avec la méthode **transform** :

```
1 X_train = scaler.transform(X_train)
2 X_test = scaler.transform(X_test)
```

4.4.1 Formule de normalisation

La formule utilisée lors de la normalisation est la suivante :

Pour chaque valeur x d'une feature :

$$x_{\text{normalisé}} = \frac{x - \mu}{\sigma}$$

où :

- μ est la moyenne de la feature calculée sur **X_train**
- σ est l'écart-type de la feature calculé sur **X_train**

Remarque cruciale : Les mêmes valeurs de μ et σ (appries sur **X_train**) sont utilisées pour normaliser à la fois **X_train** et **X_test**. Cela garantit que les deux ensembles sont normalisés de manière cohérente, comme si les données de test étaient de nouvelles données en production.

5 Définition et entraînement du modèle

5.1 Architecture du réseau de neurones

```

1 input_dim = 2  # 2 features x1 et x2
2 inputs = Input(shape=(input_dim,), name="input_layer")
3
4 # 2 couches cachées
5 x = Dense(32, activation="relu", name="dense_1")(inputs)
6 x = Dense(32, activation="relu", name="dense_2")(x)
7
8 # Couche de sortie
9 outputs = Dense(1, activation="sigmoid", name="output")(x)
10
11 model = Model(inputs, outputs, name="mlp_moons_simple")

```

5.1.1 Fonctionnement du réseau

Un réseau de neurones est composé de couches. Ici, nous avons deux couches denses avec 32 neurones chacune.

Passage des données : Les points sont passés un par un à la première couche dense. Pour chaque point, chacun des 32 neurones calcule un nombre. En sortie de la première couche dense, on obtient donc 32 nombres (de a_1 à a_{32}).

Calcul dans un neurone Chaque neurone effectue deux opérations successives :

1. Calcul linéaire :

$$z = W \cdot x + b \quad \text{équivalent à} \quad z = w_1 \cdot x_1 + w_2 \cdot x_2 + b$$

Les paramètres w_1 , w_2 et b sont différents pour chaque neurone et ce sont ces paramètres que le modèle ajuste lors de l'entraînement.

2. Activation :

$$a = \text{fonction}(z)$$

Une fonction d'activation est appliquée sur le résultat z pour chaque neurone.

Propagation à travers les couches Les 32 nombres obtenus de la première couche dense sont passés en entrée à la deuxième couche dense.

Couche Dense 2 :

Chaque neurone de la couche 2 reçoit les 32 valeurs de la couche 1 et effectue :

Calcul linéaire pour le neurone i :

$$z_i = w_{i,1} \cdot a_1 + w_{i,2} \cdot a_2 + \dots + w_{i,32} \cdot a_{32} + b_i$$

où $w_{i,1}, w_{i,2}, \dots, w_{i,32}$ sont les 32 poids spécifiques à ce neurone, et b_i son biais.

Activation :

$$\text{sortie}_i = \text{ReLU}(z_i)$$

Ce processus est répété pour les 32 neurones de la couche 2, produisant 32 valeurs en sortie.

Couche de sortie :

Ces 32 valeurs sont ensuite passées à la couche de sortie qui contient 1 seul neurone. Ce neurone effectue :

Calcul linéaire :

$$z = w_1 \cdot a_1 + w_2 \cdot a_2 + \dots + w_{32} \cdot a_{32} + b$$

Activation : Application de la fonction sigmoid qui produit une probabilité entre 0 et 1.

5.1.2 Choix des fonctions d'activation

Pour les couches cachées : ReLU Pour les couches cachées (les couches denses), on utilise la fonction d'activation **ReLU** (Rectified Linear Unit) :

$$\text{ReLU}(z) = \max(0, z)$$

Cette fonction garde le nombre s'il est positif, sinon elle le transforme en 0.

Pour la couche de sortie : Sigmoid Pour la couche de sortie, on utilise la fonction d'activation **Sigmoid** car il s'agit d'une classification binaire :

$$\text{Sigmoid}(z) = \frac{1}{1 + e^{-z}}$$

Cette fonction ramène le nombre z issu du calcul linéaire à un nombre compris entre 0 et 1 (une probabilité). Si cette probabilité est inférieure à 0.5, la prédiction finale sera 0, sinon 1.

Ajustement des paramètres La sortie (probabilité) est comparée avec `y_train`, et le modèle s'ajuste en fonction de l'erreur. Ce qui est ajusté, ce sont les poids w et les biais b de chaque neurone de chaque couche.

5.1.3 Choix de la couche Dense et notation fonctionnelle

Pourquoi la couche Dense ?

On a choisi la couche Dense car nos données d'entrée sont x_1 et x_2 qui forment un vecteur simple (données tabulaires). La couche Dense connecte tous les neurones entre les couches, ce qui est approprié pour ce type de données.

Notation fonctionnelle vs séquentielle :

Il existe deux façons de définir un modèle en Keras :

Notation séquentielle (plus simple mais moins flexible) :

```
1 model = Sequential([
2     Dense(32, activation='relu'),
3     Dense(32, activation='relu'),
4     Dense(1, activation='sigmoid')
5 ])
```

Notation fonctionnelle (plus flexible, utilisée ici) :

```
1 inputs = Input(shape=(2,))
2 x = Dense(32, activation="relu", name="dense_1")(inputs)
3 x = Dense(32, activation="relu", name="dense_2")(x)
4 outputs = Dense(1, activation="sigmoid", name="output")(x)
5 model = Model(inputs, outputs)
```

La notation fonctionnelle est privilégiée car elle est plus facile à étendre pour des architectures complexes.

5.2 Structure du modèle : `model.summary()`

On peut visualiser la structure du modèle à l'aide de `model.summary()`. L'affichage montre pour chaque couche :

- La forme de sortie (*Output Shape*)
- Le nombre de paramètres à apprendre (*Param #*)

Calcul du nombre de paramètres Couche Dense 1 : 96 paramètres

Pour chaque neurone, le modèle doit apprendre w_1 , w_2 et b , soit 3 paramètres.

$$3 \times 32 = 96 \text{ paramètres}$$

Couche Dense 2 : 1,056 paramètres

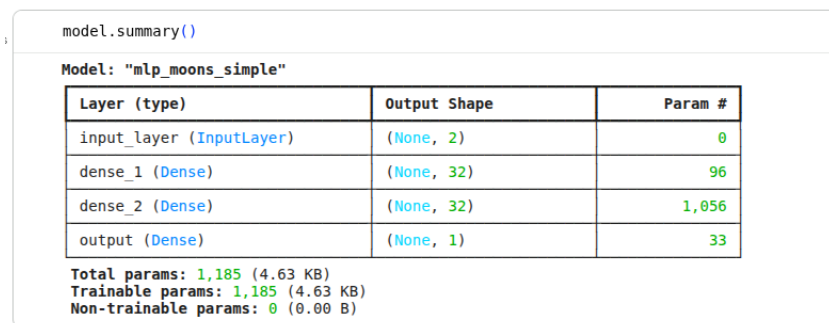
Cette couche reçoit 32 entrées (sortie de Dense 1) et contient 32 neurones. Chaque neurone doit apprendre 32 poids (un par entrée) plus 1 biais.

$$(32 + 1) \times 32 = 33 \times 32 = 1056 \text{ paramètres}$$

Couche de sortie : 33 paramètres

Cette couche reçoit 32 entrées et contient 1 neurone. Ce neurone doit apprendre 32 poids plus 1 biais.

$$32 + 1 = 33 \text{ paramètres}$$



```
model.summary()
```

Model: "mlp_moons_simple"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 2)	0
dense_1 (Dense)	(None, 32)	96
dense_2 (Dense)	(None, 32)	1,056
output (Dense)	(None, 1)	33

Total params: 1,185 (4.63 KB)
Trainable params: 1,185 (4.63 KB)
Non-trainable params: 0 (0.00 B)

FIGURE 1 – Structure du modèle affichée par `model.summary()`

5.3 Compilation du modèle

```
1 learning_rate = 1e-3
2 model.compile(
3     optimizer=Adam(learning_rate),
4     loss="binary_crossentropy",
5     metrics=["accuracy"]
6 )
```

5.3.1 Paramètres de compilation

optimizer : C'est l'algorithme qui permet d'ajuster les poids et les biais pendant l'entraînement.

Ici, on utilise **Adam** avec un **learning_rate** faible ($1e-3 = 0.001$). Un learning rate petit signifie que l'apprentissage sera lent et progressif, ce qui évite de "rater" la solution optimale en faisant des pas trop grands.

loss : C'est la fonction de perte qui permet de mesurer à quel point le modèle se trompe.

On utilise `binary_crossentropy` car notre problème est une classification binaire (2 classes).
metrics : C'est ce qu'on veut observer durant l'entraînement.
Ici, `accuracy` nous donne le pourcentage de bonnes prédictions à chaque epoch.

5.4 Entraînement du modèle : premier essai

```
1 epochs = 30
2 history = model.fit(
3     X_train, y_train,
4     epochs=epochs,
5     verbose=1
6 )
```

On commence l'entraînement avec `fit()`. On spécifie 30 epochs, c'est-à-dire que le modèle va voir 30 fois l'ensemble des données d'entraînement.

Observation : À chaque itération (epoch), on observe que :

- **accuracy augmente** : Le pourcentage de bonnes prédictions s'améliore
- **loss diminue** : L'erreur du modèle diminue

Cela confirme que le modèle apprend et devient plus performant au fil des epochs.

Les courbes affichées grâce à `plot_history_simple(history)` confirment cette progression.

`plot_history_simple(history)`

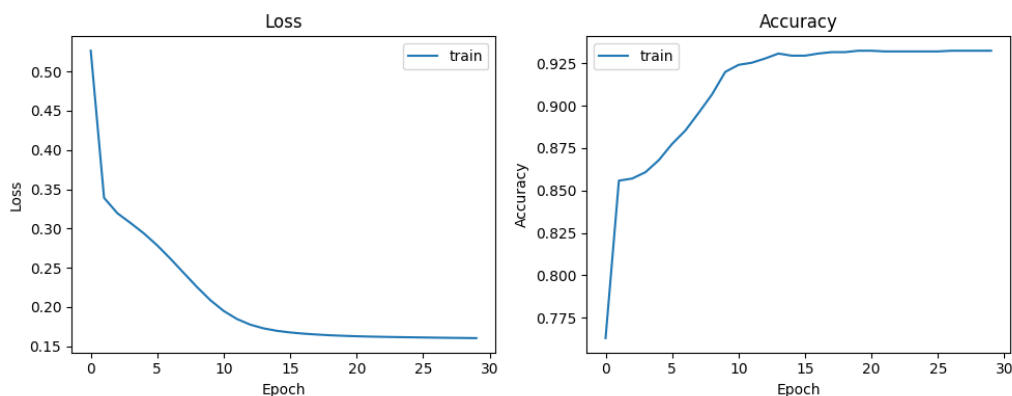


FIGURE 2 – Courbes de loss et accuracy avec 30 epochs

5.5 Test avec 10 epochs

Avec seulement 10 epochs, les résultats ne sont pas très indicateurs :

- La courbe de **loss** diminue de façon presque linéaire, on ne voit pas bien la progression
- La courbe d'**accuracy** est variable : elle diminue et augmente au fil des epochs sans tendance claire

Conclusion : 10 epochs ne suffisent pas pour un apprentissage stable.

5.6 Introduction du batch_size

```
1 batch_size = 64
2 epochs = 60
3
4 history = model.fit(
5     X_train, y_train,
6     epochs=epochs,
```

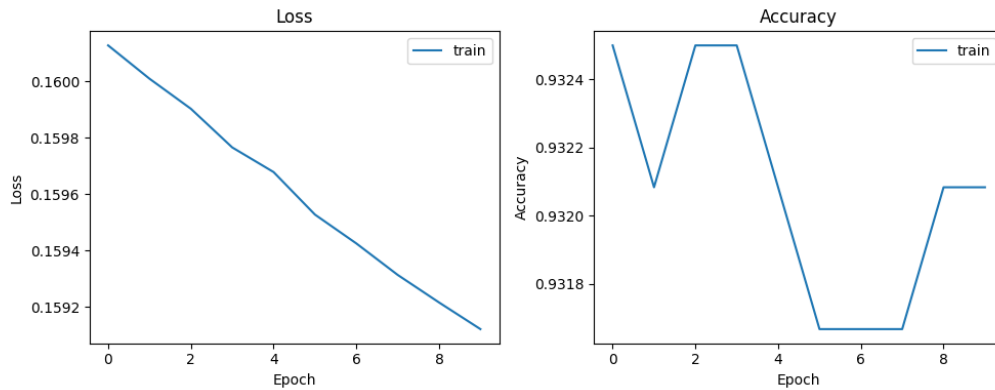


FIGURE 3 – Courbes de loss et accuracy avec seulement 10 epochs

```

7     verbose=1,
8     batch_size=batch_size
9 )

```

5.6.1 Qu'est-ce que le `batch_size` ?

Le `batch_size` détermine comment les données sont présentées au modèle pendant l'entraînement.

Sans `batch_size` spécifié : À chaque epoch, le modèle calcule le taux d'erreur sur tous les points avant de mettre à jour les poids (une seule mise à jour par epoch).

Avec `batch_size = 1` : Le modèle met à jour les poids après chaque point donné en entrée. Cela est très coûteux en mémoire RAM et lent.

Avec `batch_size = 64` : Le modèle calcule l'erreur sur des paquets de 64 points, puis met à jour les poids. Les données sont donc traitées par groupes de 64.

Valeurs couramment utilisées : 8, 16, 32, 64. Les plus populaires sont 32 et 64.

5.6.2 Impact du `batch_size` sur les courbes

En variant les valeurs de `batch_size`, les graphes de loss et accuracy changent :

Batch_size petit (8, 16) :

- Mises à jour fréquentes des poids
- Courbes avec beaucoup de fluctuations (zigzag)
- Apprentissage plus "nerveux"

Batch_size moyen (32, 64) :

- Bon équilibre entre vitesse et stabilité
- Graphes lisibles avec une progression claire
- On voit la loss qui diminue progressivement
- On voit l'accuracy qui augmente progressivement

Batch_size grand (128, 256) :

- Mises à jour moins fréquentes
- Courbes très lisses
- Plus rapide (moins d'itérations)

Avec `batch_size=64`, les graphes sont particulièrement lisibles et on voit clairement la progression du modèle.

5.7 Prédiction et évaluation

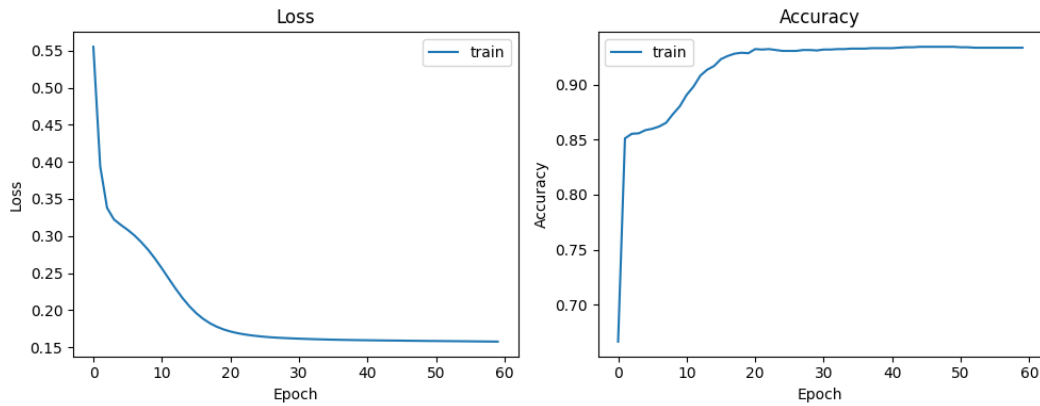


FIGURE 4 – Courbes de loss et accuracy avec `batch_size=64` et 60 epochs

```

1 y_pred = model.predict(X_test)
2
3 # Affichage d'une valeur pr dite
4 print(y_pred[1])
5
6 # Affichage du classification report
7 print(classification_report(y_test, y_pred,
8                             target_names=["class 0", "class 1"],
9                             digits=3))

```

5.7.1 Problème rencontré

L'exécution de ce code produit une erreur. La raison : `model.predict()` retourne des **probabilités** comprises entre 0 et 1, alors que `classification_report()` attend exactement des valeurs 0 ou 1.

5.7.2 Solution : conversion probabilité → classe

On doit convertir les probabilités en classes binaires (0 ou 1) :

```

1 y_prob = model.predict(X_test, verbose=0).ravel()
2 y_pred = (y_prob >= 0.5).astype(int)

```

Explication :

1. `model.predict(X_test, verbose=0).ravel()`
 - Prédit les probabilités pour chaque point de test
 - `.ravel()` : Aplatit le tableau en 1D
 - Résultat : `[0.87, 0.23, 0.91, ...]`
2. `(y_prob >= 0.5)`
 - Compare chaque probabilité au seuil 0.5
 - Donne un tableau de booléens : `[True, False, True, ...]`
3. `.astype(int)`
 - Convertit les booléens en entiers 0/1
 - Résultat : `[1, 0, 1, ...]`

Maintenant, `classification_report()` peut fonctionner correctement et afficher les métriques de performance du modèle.

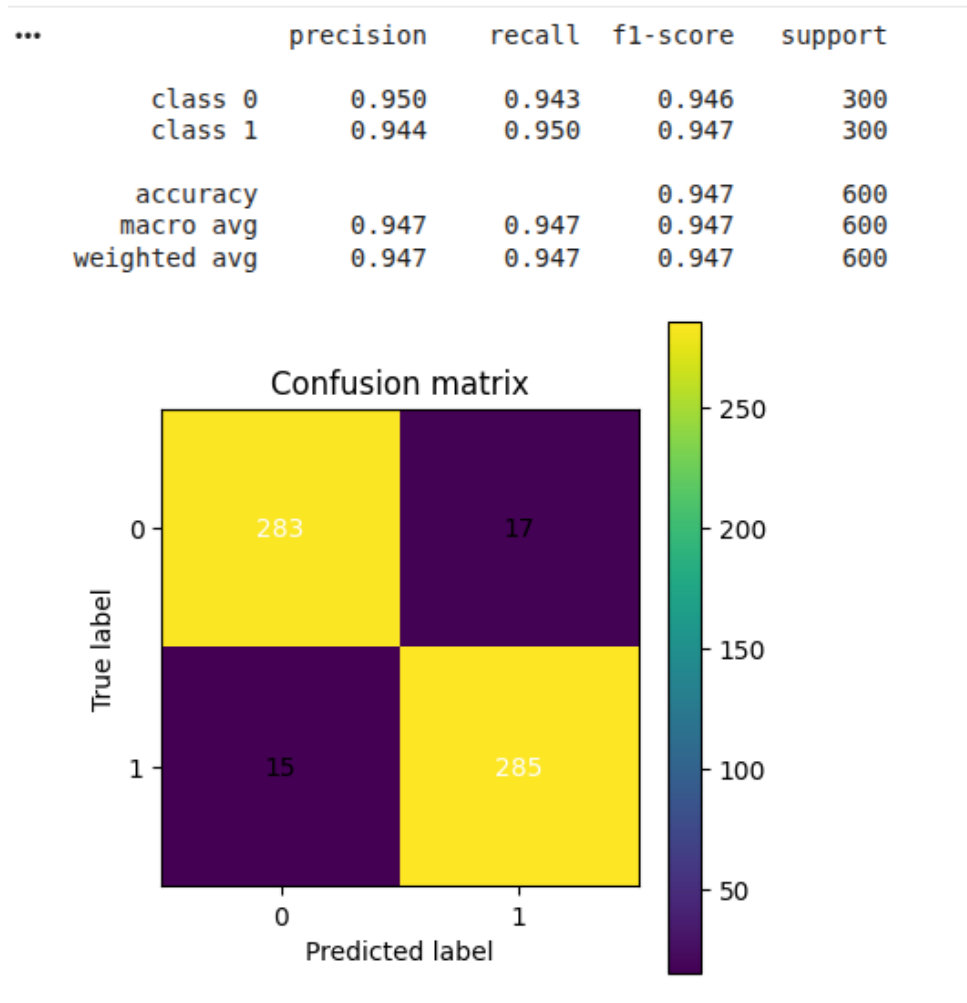


FIGURE 5 – Rapport de classification sur les données de test